

COMP 333
Fall 2026
Midterm Practice Exam #1 (Solutions)

This is representative of the kinds of topics and kind of questions you may be asked on the midterm. This practice exam and in-class handout on Java are intended to be comprehensive of everything on the exam. That is, I will not ask anything that's not somehow covered by those sources.

You are permitted to bring two 8.5 x 11 sheets of paper into the exam with you, containing either printed or handwritten notes. Both sides of both sheets can be used.

Virtual Dispatch - Conceptual Understanding

1.) Name one reason why someone might want to use virtual dispatch (i.e., Java's way of implementing ad-hoc polymorphism).

Non-exhaustive list of possibilities:

- Allows for the same method call to have different behaviors at runtime, which is more flexible
- Allows for abstraction over code behavior. Calling code only needs to know the signature of a method, and the runtime behavior can be dynamically changed by changing the underlying object the method is called on.
- Improves code modularity. Each distinct behavior can be isolated from each other behavior, and behaviors generally don't need to know about each other.

2.) Name one reason why someone might **not** want to use virtual dispatch (i.e., Java's way of implementing ad-hoc polymorphism).

Non-exhaustive list of possibilities:

- Code can become more bloated. Usually, multiple classes have to be introduced, and these all have their own boilerplate associated with them.
- Code's behavior can become less explicit, and potentially more difficult to reason about. For example, an if/else clearly indicates a condition to test and code to execute depending on the condition. However, a method call may *implicitly* do something similar.
- It generally has worse performance than an explicit if/else. (Beyond what I'd expect: behind the scenes, indirect jumps are often needed, and these can easily jump out of cached instructions. Moreover, it's difficult to predict where an indirect jump will go.)

Virtual Dispatch in Java

3.) Consider the following Java code:

```
public interface I1 {
    public void doThing();
}
public class C1 implements I1 {
    public void doThing() { System.out.println("c1"); }
}
public class C2 implements I1 {
    public void doThing() { System.out.println("c2"); }
}
public class Main {
    public static void makeCall(I1 value) {
        value.doThing();
    }
    public static void main(String[] args) {
        I1 t1 = new C1();
        I1 t2 = new C2();
        makeCall(t1);
        makeCall(t2);
    }
}
```

What is the output of the `main` method above?

c1
c2

4.) Consider the following incomplete Java code and output:

```
public class Incomplete {
    public static void printResult(final Runner r, final int i) {
        boolean result = r.someMethod(i);
        System.out.println(result);
    }
    public static void main(final String[] args) {
        final IsEven even = new IsEven();
        printResult(even, 3); // prints false
        printResult(even, 4); // prints true
        final IsLessThan ltFive = new IsLessThan(5);
        printResult(ltFive, 4); // prints true
        printResult(ltFive, 6); // prints false
        final IsLessThan ltZero = new IsLessThan(0);
        printResult(ltZero, -1); // prints true
        printResult(ltZero, 1); // prints false
    }
}
```

The only built-in Java types or methods this code uses are `boolean`, `int`, `main`, and `System.out.println`. Define any interfaces and/or classes necessary to make the output in the comments work. You should not have to modify any code here. Multiple answers are possible.

```
public interface Runner {
    public boolean someMethod(int i);
}
public class IsEven implements Runner {
    public boolean someMethod(int i) {
        return i % 2 == 0;
    }
}
public class IsLessThan implements Runner {
    public final int value;
    public IsLessThan(final int value) {
        this.value = value;
    }
    public boolean someMethod(int i) {
        return i < value;
    }
}
```

5.) Consider the following Java code, which simulates a lock which can be either locked or unlocked. The lock is an immutable data structure, so locking or unlocking returns a new lock in an appropriate state. `boolean` and `System.out.println` are built-in.

```
public class Lock {
    private final boolean locked;

    public Lock(final boolean locked) {
        this.locked = locked;
    }

    public Lock unlock() {
        if (locked) {
            System.out.println("lock unlocked");
            return new Lock(false);
        } else {
            System.out.println("lock already unlocked");
            return this;
        }
    }

    public Lock lock() {
        if (!locked) {
            System.out.println("lock locked");
            return new Lock(true);
        } else {
            System.out.println("lock already locked");
            return this;
        }
    }

    public boolean isLocked() {
        return locked;
    }
}
```

On the next page, refactor this code to use virtual dispatch (i.e., Java's way of implementing ad-hoc polymorphism), instead of using `if/else`. Your modified version should work with the `example` method provided below.

```
public static void example() {
    Lock lock0 = new UnlockedLock();
    Lock lock1 = lock0.unlock();
    Lock lock2 = lock1.lock();
    System.out.println(lock2.isLocked()); // prints true
}
```

```
public interface Lock {
    public Lock unlock();
    public Lock lock();
    public boolean isLocked();
}

public class UnlockedLock implements Lock {
    // this constructor is not strictly needed; Java can
    // make it for us
    public UnlockedLock() {}

    public Lock unlock() {
        System.out.println("lock already unlocked");
        return this;
    }

    public Lock lock() {
        System.out.println("lock locked");
        return new LockedLock();
    }

    public boolean isLocked() {
        return false;
    }
}

public class LockedLock implements Lock {
    // this constructor is not strictly needed; Java can
    // make it for us
    public LockedLock() {}

    public Lock unlock() {
        System.out.println("lock unlocked");
        return new UnlockedLock();
    }

    public Lock lock() {
        System.out.println("lock already locked");
        return this;
    }

    public boolean isLocked() {
        return true;
    }
}
```

6.) Consider the following Java code, which shows expected output in the comments:

```
public class Concatter {
    private final boolean leftThenRight;
    public Concatter(final boolean leftThenRight) {
        this.leftThenRight = leftThenRight;
    }
    public String concat(String a, String b) {
        if (leftThenRight) {
            return a + b;
        } else {
            return b + a;
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Concatter leftRight = new Concatter(true);
        Concatter rightLeft = new Concatter(false);

        String res1 = leftRight.concat("foo", "bar");
        System.out.println(res1); // prints foobar

        String res2 = rightLeft.concat("foo", "bar");
        System.out.println(res2); // prints barfoo
    }
}
```

`boolean`, `String`, `main`, and `System.out.println` are built-in in Java.

`Concatter` objects can be in one of two states: concatenating input strings from left to right (`leftThenRight` is `true`), or concatenating input strings from right to left (`leftThenRight` is `false`). The state cannot be changed after the `Concatter` objects is created (`leftThenRight` is marked `final`). The `concat` method performs the concatenation over its inputs `a` and `b`.

On the next page, refactor this code to introduce separate `LeftRight` and `RightLeft` classes. Each of these classes specializes `concat`'s behavior, and do not need `if`, ternary, or any other conditional execution. You will need to redefine `Concatter` itself, in addition to defining your `LeftRight` and `RightLeft` classes. Your revised version should work correctly with the provided `RevisedMain` class on the next page.

```
public class RevisedMain {  
    public static void main(String[] args) {  
        Concatter leftRight = new LeftRight();  
        Concatter rightLeft = new RightLeft();  
  
        String res1 = leftRight.concat("foo", "bar");  
        System.out.println(res1); // prints foobar  
  
        String res2 = rightLeft.concat("foo", "bar");  
        System.out.println(res2); // prints barfoo  
    }  
}
```

```
public interface Concatter {  
    public String concat(String a, String b);  
}
```

```
public class LeftRight implements Concatter {  
    public String concat(String a, String b) {  
        return a + b;  
    }  
}
```

```
public class RightLeft implements Concatter {  
    public String concat(String a, String b) {  
        return b + a;  
    }  
}
```

7.) Consider the following code snippet:

```
public class Main {  
    public static void main(String[] args) {  
        Operation op1 = new AddOperation();           // line 3  
        Operation op2 = new SubtractOperation();      // line 4  
        int res1 = op1.doOp(5, 3);                    // line 5  
        int res2 = op2.doOp(5, 3);                    // line 6  
        System.out.println(res1); // line 7; should print 8  
        System.out.pritnln(res2); // line 8; should print 2  
    }  
}
```

Define any interfaces and/or classes necessary to make this snippet print 8, followed by 2. main, String, int, and System.out.println are built-in in Java.

```
// From lines 3-4, we know that Operation must be a superclass  
// of AddOperation and SubtractOperation, based on the types of  
// op1 and op2. From line 5, we know that Operation must have a  
// doOp method, that it must return an int, and that it must  
// take two ints. From line 3, 5, and 7, we can infer that  
// AddOperation's doOp must be adding its arguments, and  
// similarly from lines 4, 6, and 8, we can infer  
// SubtractOperation's doOp must be subtracting its arguments.
```

```
public interface Operation {  
    public int doOp(int first, int second);  
}  
  
public class AddOperation implements Operation {  
    public int doOp(int first, int second) {  
        return first + second;  
    }  
}  
  
public class SubtractOperation implements Operation {  
    public int doOp(int first, int second) {  
        return first - second;  
    }  
}
```

Types

8.) The code below does not compile. Why?

```
public interface MyInterface {
    public void foo();
}

public class MyClass implements MyInterface {
    public void foo() {}
    public void bar() {}

    public static void main(String[] args) {
        MyInterface a = new MyClass();
        a.bar();
    }
}
```

The bar method is only available on MyClass, not MyInterface. a is of type MyInterface, so the bar method is not available, even though the runtime type of a will be MyClass (which does have bar).

9.) The code below does not compile. Why?

```
public class Foo {
    public Foo() {}
}

public class Bar extends Foo {
    public Bar() {}
}

public class Problem1 {
    public static void main(String[] args) {
        Foo a = new Foo();
        Foo b = new Bar();
        Bar c = new Foo();
        Bar d = new Bar();
    }
}
```

Foo is not a subtype of Bar, so you can't do Bar c = new Foo().

10.) Java supports subtyping polymorphism. Write a Java code snippet that compiles and uses subtyping polymorphism.

Using the class definitions from the prior problem:

```
Foo b = new Bar();
```

11.) The code below does not compile. Why?

```
public interface Base {}

public class Sub implements Base {
    public Sub() {}
}

public class Problem2 {
    public static void doSomething(Sub s) {}

    public static void main(String[] args) {
        Base b = new Sub();
        doSomething(b);
    }
}
```

Base is not a subtype of Sub. doSomething expects a Sub or a subtype of Sub, but we have given it b, which has a compile-time type of Base. Even though the runtime type of b is Sub, the compiler only considers compile-time types to determine if something compiles.

12.) Consider the method below, using Base and Sub from the prior problem. Assume randomBoolean returns a randomly-selected boolean value at runtime.

```
public void example() {
    Base foo = (randomBoolean()) ? new Base() : new Sub();
}
```

What is the compile-time type of foo? What are the possible runtime types of foo?

Compile-time type: Base (this refers to the type explicitly put in the program)

Runtime types: Base and Sub (this refers to the types of the values actually assigned)